Databases and SQL

DATA201/422 Data Wrangling

John Bryant

2026-09-07

Who is this? Where is Kat?

- John Bryant
- Statistical demographer
  - Population forecasts, life expectancy, migration measurement, etc
  - Worked at Treasury, Stats NZ, consultancies, universities in Thailand
  - Currently at Bayesian Demography Limited
- Computing background
  - Mainly R, but also Python (a little rusty), UNIX tools, C, LaTeX, Quarto, make, etc
  - Reproducible research
  - Statistical production processes

Topics for lectures on databases and SQL

- Monday: Introduction to databases and SQL
  - What are databases?
  - What is SQL?
  - Databases in the data science pipeline
  - Intro to SQL
- Tuesday: More database topics
  - Normalisation
  - Modular SQL
  - AI and SQL

Databases

Databases and Database Management Systems

- A database is any organised set of data
- A Database Management System DBMS is software for managing databases
  - SQL Server, Oracle, MySQL, PostgreSQL, SQLite, DuckDB, etc
  - In practice, in computer science ‘database’ can mean DBMS
- Most common: ‘relational’ database, operated using SQL language
  - This is what we’ll look at
- Lots of exotic alternatives
  - Special databases for multiway arrays, spatial data, unstructured data, etc
  - Can be useful for data science

Data warehouse

- Most databases are *not* optimised for analysis
  - Optimised for transaction processing, eg absorbing new data quickly and safely
  - Code to analyse data can be complex and slow
- Data warehouses: databases that *are* optimised for analysis
  - Organised to make analysis queries simpler and faster
  - Often centralises data from multiple transactional databases
- Lots of different designs
  - Enterprise warehouse, dimensional warehouse, etc
  - Each has different trade-offs between flexibility, speed, consistency, reliability

Examples of data science tasks that SQL databases do well

- Simple validation
  - E.g. checking that values lie within bounds
- Simple filtering
  - E.g. only show transactions from January 2026 onwards
- Tabulation
  - E.g. summing expenditures by category
- Text or date operations
  - E.g. converting `"7/7/26"` to `"2026-07-07"`
- Sorting
  - E.g. ordering transactions by date and time
- Basic maths
  - E.g. adding, subtracting, minimum, maximum

Examples of data science tasks that SQL databases do badly

- Fitting statistical models
- Working with data structures that are not data frame-style tables
  - E.g. tree structures, multiway arrays
- Complex cleaning tasks
  - E.g. complex decision processes

ASIDE

Data structures

- Key idea in data science, computer science
- Examples of data structures in data science
  - Time series
  - Networks (e.g. Facebook social graph)
  - Multiway arrays
  - Panel/longitudinal data
- Finding appropriate data structure(s) is crucial!
  - Does your data wrangling and modelling feels messy or awkward? You might be using the wrong data structures

Summary: Databases in the data science pipeline

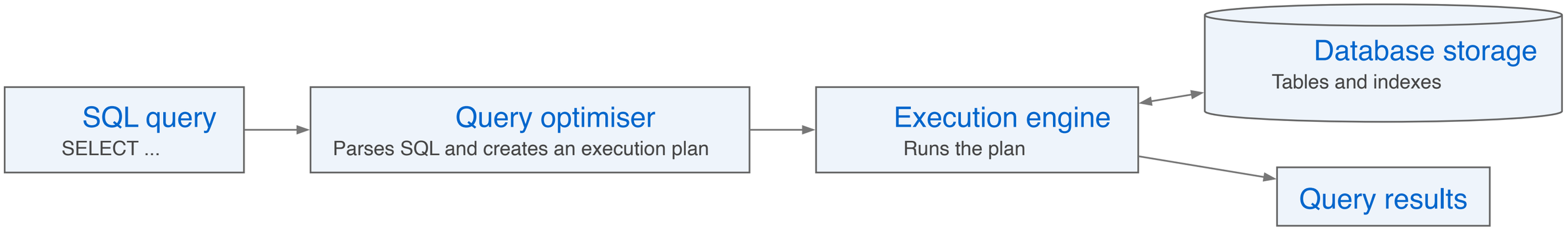
Task	Typical tool
Deterministic, bulk data processing	Database
Probabilistic, complex data processing	R/Python
Statistical modelling, machine learning	R/Python

SQL

What is SQL?

- Language for working with databases
  - Adding new data
  - Extracting existing data
- Surprisingly hard to improve on
  - Dates from 1970s
  - People have been proposing replacements since then
  - But SQL survives
- Lots of dialects
  - SQL that is valid on one database may not be valid on another

Under the hood



- SQL is “declarative”
  - What you want, not how to get it
  - But optimiser can still be quite dumb

Example

Data

products

id	name	price
1	Apple	3.50
2	Bread	7.00
3	Pear	3.00
4	Milk	2.50
5	Eggs	4.00

SQL

```
1 SELECT id, name, price
2 FROM products
3 WHERE price > 3
```

Results

id	name	price
1	Apple	3.50
2	Bread	7.00
5	Eggs	4.00

Another Example

Data

products

id	name	price
1	Apple	3.50
2	Bread	7.00
3	Pear	3.00
4	Milk	2.50
5	Eggs	4.00

SQL

```
1  SELECT name, price
2  FROM products
3  WHERE price < 6
4  AND name != 'Milk'
5  ORDER BY price
```

Results

name	price
Pear	3.00
Apple	3.50
Eggs	4.00

Row order in relational databases

- In pure SQL, the row order of results is unspecified
  - ⚠️ Different from data frames in Python and R
- Some databases do order rows in some circumstances
  - But can be dangerous to rely on this (Why?)
- If you need ordering, use `ORDER BY`

```sql
SELECT name, price
FROM products
WHERE price < 6
AND name != 'Milk'
ORDER BY price
```

Aggregation

Data

costs

type	cost
B	10
A	20
A	30

SQL

```
1 SELECT
2   type,
3   SUM(cost) AS total
4 FROM costs
5 GROUP BY type
6 ORDER BY type
```

Results

type	total
A	50
B	10

Inner Join

Data

products

id	name
1	Apple
2	Bread
3	Pear
4	Milk
5	Eggs

ratings

name	rating
Apple	yum
Bread	yuck
Milk	yuck
Pear	yum

SQL

```
1  SELECT
2    products.id,
3    products.name,
4    ratings.rating
5  FROM products
6  INNER JOIN ratings
7  ON products.name
8    = ratings.name
```

Results

id	name	rating
3	Pear	yum
1	Apple	yum
4	Milk	yuck
2	Bread	yuck

Left Join

Data

products

id	name
1	Apple
2	Bread
3	Pear
4	Milk
5	Eggs

ratings

name	rating
Apple	yum
Bread	yuck
Milk	yuck
Pear	yum

SQL

```sql
SELECT
  products.id,
  products.name,
  ratings.rating
FROM products
LEFT JOIN ratings
ON products.name
  = ratings.name
```

Results

id	name	rating
3	Pear	yum
1	Apple	yum
4	Milk	yuck
2	Bread	yuck
5	Eggs	NULL

The `*` symbol

- If table `products` has columns `id`, `name`, and `price`, then

```
1 SELECT *
2 FROM products
```

gives the same result as

```
1 SELECT id, name, price
2 FROM products
```

- The `*` form is OK for interactive use, but not for production code
- **Why?**

Using SQL from within R/Python

```python
import sqlite3
import pandas as pd

sales = pd.DataFrame(
    {
        "product": ["Laptop", "Monitor", "Laptop"],
        "region": ["North", "North", "South"],
        "amount": [1200.00, 400.00, 1150.00],
    }
)

connection = sqlite3.connect("sales.db")

sales.to_sql(
    "sales",
    connection,
    if_exists="replace",
    index=False,
)
```

continued on next slide…

Using SQL from within R/Python

…continued from previous slide

```
query = """
    SELECT product, SUM(amount) AS total_sales
    FROM sales
    WHERE region = ?
    GROUP BY product
"""

results = pd.read_sql_query(
    query,
    connection,
    params=("North",),
)

connection.close()
```

NOTE - R also has `dbplyr`

Strengths and limitations of SQL

- Strengths
  - Elegant descriptions of fundamental data operations
  - Scales well - can cope with huge datasets
- Limitations
  - Complex queries are hard to understand and test
  - Limited data structures